Algorithms assignment 2

Thobias K. Høivik

For November 13, 2026

Problem (2 Problem 5)**.** Given 7 jobs with corresponding processing times,

Job j	1	2	3	4	5	6	7
Processing Time p_j	4	7	2	9	12	5	3

The jobs will be processed on 3 identical machines without interruption. All jobs are avliable at time 0. (Model on slide 13, Chapter 2)

a) Use the local search algorithm where we have all jobs on machine 1 in the beginning (in the same order as they are given). When you move a job, move it to the machine it can start earliest. I there is a tie, use the machine with the lowest number. What is the result, both time when the last job finish and the actual schedule?
b) Use the Greedy algorithm on the same data. When have all jobs finished?
c) Use the Longest processing time rule. When have all jobs finished?
d) Give a lower bound on the best solution and give a short explanation?
e) Is it possible to achieve the answer in d) (Explain)?

This is the classical identical parallel-machine makespan minimization problem, denoted

$$P_3 \| \, C_{\max}.$$

We want to find a schedule minimizing

$$C_{\max} = \max_{i \in [3]} L_i,$$

where

$$L_i := \sum_{j \text{ assigned to machine } i} p_j$$

is the total processing time on machine i.

Solution to (a). The initial schedule gives all seven jobs to machine 1 in the described order by

$$M_1 = \underbrace{1}_{4} \quad \underbrace{1}_{4} \quad \underbrace{2}_{7} \quad \underbrace{3}_{2} \quad \underbrace{4}_{9} \quad \underbrace{5}_{12} \quad \underbrace{6}_{5} \quad \underbrace{7}_{3}.$$

Note: This accidentally looks a lot like fractions, but I am just illustrating the processing times p_j.

We get initial loads

$$(L_1, L_2, L_3) = (42, 0, 0)$$

with $C_{\max} = 42$.

If M_j is the unique machine for which $L_j = C_{\max}$ we find that it is advantageous to swap if

$$L_j + c < L_i,$$

or equivalently

$$c < L_i - L_j.$$

We perform the following procedure.

Require: Jobs J with processing times p_j, machines $M_1, \dots, M_m$
Ensure: A locally optimal schedule

```
Assign all jobs to M_1 in their given order
L_i ← Σ_{j∈M_i} p_j for each i ∈ {1, ..., m}
while true do
    i ← arg max_k L_k
    ℓ ← last job on M_i
    j ← arg min_{k≠i} L_k
    if L_j + p_ℓ ≥ L_i then
        return (M_1, ..., M_m)
    end if
    Remove ℓ from M_i
    Append ℓ to M_j
    L_i ← L_i − p_ℓ
    L_j ← L_j + p_ℓ
end while
```

So here we begin with

$$(42, 0, 0).$$

The job which finishes last is $j = 7$ so we get

$$(42, 0, 0) \mapsto (39, 3, 0).$$

Now M_1 still is the machine with maximum load so we take $j = 6$, and move

$$(39, 3, 0) \mapsto (34, 3, 5).$$

Repeat to get

$$(34, 3, 5) \mapsto (22, 15, 5) \mapsto (15, 15, 14).$$

Now we look at M_1 again and see that for $j = 3$ we have $p_j = 3 < L_3 = 14$ so the algorithm halts with

$$C_{\max} = 14 \leq 2C^*_{\max} = 14 \cdot 2 = 28$$

as expected.

Running time is $O(nm)$.

■

Solution to (b). We perform

Require: Jobs with processing times $p_1, \ldots, p_n$ and m machines

1: $L_i \leftarrow 0$ for all $i = 1, \ldots, m$
2: **for** $j = 1, \ldots, n$ **do**
3: $\quad i^* \leftarrow \arg\min_i L_i$
4: $\quad$ Assign job j to machine M_{i^*}
5: $\quad L_{i^*} \leftarrow L_{i^*} + p_j$
6: **end for**
7: **return** $\max_i L_i$

We begin at $j = 1$ and since all M_i have $L_i = 0$ we get L_i's

$$(4, 0, 0).$$

Next

$$(4, 7, 0),$$

then

$$(4, 7, 2).$$

Once again L_3 is the minimum so next pass gives

$$(4, 7, 11).$$

Now L_1 is smallest so we get

$$(16, 7, 11).$$

Now L_2 is smallest so we get

$$(16, 12, 11).$$

Now L_3 is smallest so we get

$$(16, 12, 14).$$

We obtain $C_{\max} = 16$ with the algorithm running in time $O(nm)$. ■

Solution to (c). We perform

Require: Jobs J with processing times p_j, m identical machines

1: Sort jobs by nonincreasing processing time
2: $L_i \leftarrow 0$ for all $i = 1, \ldots, m$
3: **for** each job j in sorted order **do**
4: $\quad i^* \leftarrow \arg\min_i L_i$
5: $\quad$ Assign job j to machine M_{i^*}
6: $\quad L_{i^*} \leftarrow L_{i^*} + p_j$
7: **end for**
8: **return** $\max_i L_i$

Performiing the algorithm we get

$$\begin{aligned}(12,0,0) &\mapsto (12,9,0)\\ &\mapsto (12,9,7)\\ &\mapsto (12,9,12)\\ &\mapsto (12,13,12)\\ &\mapsto (15,13,12)\\ &\mapsto (15,13,12)\end{aligned}$$

attaining a maximum of 15.

Running time is $O(n \log n + mn)$ since we sort first. ■

Solutioin and proof sketch for (d). Let $C^*_{\max}$ denote the optimal makespan, and let L_i be the total processing time assigned to machine M_i.

Since every job must be processed, we have

$$\sum_{i=1}^{m} L_i = \sum_{j=1}^{n} p_j.$$

Moreover, since each machine processes jobs sequentially,

$$C^*_{\max} \geq \max_i L_i \geq \frac{1}{m} \sum_{i=1}^{m} L_i = \frac{1}{m} \sum_{j=1}^{n} p_j.$$

We also necessarily have

$$C^*_{\max} \geq \max_j p_j,$$

since every job must finish before the schedule is complete.

Combining we get

$$C^*_{\max} \geq \max\left\{ \frac{1}{m} \sum_{j=1}^{m} p_j,\ \max_j p_j \right\}.$$

■

Solution to (e). Consider the following schedule,

$$\begin{aligned}M_1 &: \{5,3\}, L_1 = 12 + 2 = 14\\ M_2 &: \{4,6\}, L_2 = 9 + 5 = 14,\\ M_3 &: \{2,1,7\} L_3 = 7 + 4 + 3 = 14.\end{aligned}$$

This is a feasible schedule with

$$C_{\max} = \max\{14,14,14\} = 14 = C^*_{\max}.$$

Problem (2, Problem 6)**.** The adjacency matrix of a graph on 6 vertices is given below. The triangle inequality holds for the whole graph.

$$\begin{bmatrix} 0 & 4 & 5 & 10 & 11 & 7 \\ 4 & 0 & 9 & 6 & 7 & 9 \\ 5 & 9 & 0 & 9 & 6 & 2 \\ 10 & 6 & 9 & 0 & 3 & 9 \\ 11 & 7 & 6 & 3 & 0 & 6 \\ 7 & 9 & 2 & 9 & 6 & 0 \end{bmatrix}$$

a) Verify that the triangle inquality holds when going from vertex 1 to 5.
b) Run the nearest addition algorithm on the graph. If there are more than one candidate to add in a step, use the vertex with the smallest number. It is sufficient to have fours nodes in the tour (skipping the last two iterations through the loop).
c) Run the Double tree algorithm on the graph. If there are more than one minimum spanning tree, you should run it on every tree.
d) Run Christofides' algorithm on the graph. If there are more than one minimum spanning tree, you should run it on every tree.

Let $G = ([6], E) \cong K_6$ be the complete graph (undirected) on 6 vertices with edge weights given in the adjacency matrix in the problem description.

Denote the weights c_{ij}. I.e. $c_{12} = 4,\ c_{15} = 11$ and so forth.

Solution for (a). We have $c_{15} = 11$. The possibilities are

$$\begin{aligned} c_{12} + c_{25} &= 4 + 7 = 11, \\ c_{13} + c_{35} &= 5 + 6 = 11, \\ c_{14} + c_{45} &= 10 + 3 = 13, \\ c_{16} + c_{65} &= 7 + 6 = 13. \end{aligned}$$

Thus

$$c_{15} = 11 \leq \min\{11, 11, 13, 13\} = 11$$

so we're good.

Solution to (b). We wish to find a permutation π of the vertices minimizing

$$c_{\pi(1)\pi(2)} + c_{\pi(2)\pi(3)} + \cdots + c_{\pi(6)\pi(1)}.$$

I.e. we want to find a minimum-weight Hamiltonian cycle. The additional assumption is that the weights satisfy the triangle inequality

$$c_{ij} \leq c_{ik} + c_{ki},\ \forall i, j, k \in V = [6].$$

We begin with the two closest vertices and repeatedly add the vertex outside the tour that is closest to any vertex already in the tour.

When inserting a new vertex j between two consecutive vertices u, v, the increase in tour cost is

$$\Delta = c_{uj} + c_{jv} - c_{uv}.$$

Iteration 0: Init.

The smallest edge weight is

$$c_{36} = 2.$$

Thus, the initial tour is

$$T_0 = (3, 6, 3),$$

with cost 4.

Iteration 1.

For each unvisited vertex v, compute its distance to $S = \{3, 6\}$,

$$d(1, S) = 5$$
$$d(2, S) = 9$$
$$d(4, S) = 9$$
$$d(5, S) = 6.$$

Therefore, we choose vertex 1, whose nearest vertex is 3. We insert it into the tour to get

$$T_1 = (3, 1, 6, 3).$$

The cost is $w(T_1) = 14$.

Iteration 2.

Repeat to find $\arg\min_v d(v, S) = 2$ with weight 4. We have to places we can set it. If we insert it between 3 and 1 we get

$$\Delta_1 = c_{32} + c_{21} - c_{31} = 8.$$

If we choose 1 and 6 we get

$$\Delta_2 = 6.$$

Thus we choose it such that

$$T_2 = (3, 1, 2, 6, 3).$$

Now the cost is $w(T_2) = 20$. For the grader,

$$\text{weight is } \boxed{20}$$

Solution to (c). The four smallest edge weights are

$$c_{36} = 2,$$
$$c_{45} = 3,$$
$$c_{12} = 4,$$
$$c_{13} = 5.$$

Kruskal's algorithm selects all four, giving two connected components in

$$A = \{1, 2, 3, 6\},\ B = \{4, 5\}.$$

The minimum weight of an edge joining these components is 6, attained by the edges

$$\{2, 4\},\ \{3, 5\},\ \{5, 6\}.$$

Formally, defininig

$$F = \{\{3, 6\},\ \{4, 5\},\ \{1, 2\},\ \{1, 3\}\},$$

the three MSTs are

$$T_A = F \cup \{\{2, 4\}\}$$
$$T_B = F \cup \{\{3, 5\}\}$$
$$T_C = F \cup \{\{5, 6\}\}.$$

Each has total weight

$$w(T_i) = 20.$$

Every edge in each MST is doubled, making every vertex degree even. Hence each resulting multigraph admits an Eulerian circuit.

We start at vertex 1 in each case, traversing tree branches in increasing order of vertex number.

For T_A, an Eulerian circuit is

$$W_A = (1, 2, 4, 5, 4, 2, 1, 3, 6, 3, 1).$$

Shortcutting repeated vertices yields

$$H_A = (1, 2, 4, 5, 3, 5, 1).$$

Its cost is

$$w(H_A) = 4 + 6 + 3 + 6 + 2 + 7 = 28.$$

For T_B take

$$W_B = (1, 2, 1, 4, 5, 4, 5, 3, 6, 3, 1).$$

Shortcutting gives

$$H_B = (1, 2, 3, 5, 4, 6, 1),$$

with cost

$$w(H_B) = 38.$$

Finally, for T_C, take

$$W_C = (1, 2, 1, 3, 6, 5, 4, 5, 6, 3, 1).$$

As before,

$$H_C = (1, 2, 3, 6, 5, 4, 1),$$

with

$$w(H_C) = 34.$$

All three satisfy

$$w(H_i) \leq 2w(T_i) = 40,$$

very cool. ■

Solution to (d). We add a minimum-weight perfect matching on the graph's odd-degree vertices. Consider the same three MSTs.

The edges of T_A form a path

$$5 - 4 - 2 - 1 - 3 - 6.$$

The only odd-degree vertices are endpoints $O_A = \{5, 6\}$, with only one possible perfect matching

$$P_A = \{\{5, 6\}\},\ w(P_A) = 6.$$

Adding this edge creates an Eulerian cycle, already Hamiltonian

$$H_A = (1, 2, 4, 5, 6, 3, 1),$$

with cost 26.

We repeat the procedure to produce

MST	Odd-degree vertices	Matching cost	Tour cost
T_A	$\{5, 6\}$	6	26
T_B	$\{2, 3, 4, 6\}$	8	28
T_C	$\{2, 4\}$	6	26

■

Problem (3, Problem 2)**.** I am too lazy to rewrite the entire table.

a) Run the dynamic programming algorithm (Chapter 3, slide 6) on the following kanpsack instance with $B = 10$. Show $A(i)$ twice (before and after removing the dominating pairs) for every iteration through the loop.

b) What is the maximum number of elements in $A(i)$, expressed in terms of B?

c) Let n be the number of items in the knapsack problem and let v_i be the value of item i. Furthermore let $V = \sum_{i=1}^{n} v_i$. What is the maximum of elements in $A(i)$

> expressed in terms of V?

Recall that we want to maaximize

$$\sum_{i=1}^{6} yv_i x_i$$

such that

$$\sum_{i=1}^{6} s_i x_i \leq 10,\ x_i \in \{0,1\}.$$

We define $A(i) \subseteq \mathbb{N}_0 \times \mathbb{N}_0$ where each pair $(s,v) \in A(i)$ represents a feasible choice of items from $[i]$ having total size s and value v.

We denote

$$A'(i) = A(i-1) \cup \{(s+s_i, v+v_i) \mid (s,v) \in A(i-1),\ s+s_i \leq B\}.$$

Solution to (a).

Iteration 1.

We begin with

$$A(0) = \{(0,0)\}.$$

Taking or not taking the first item gives

$$A'(1) = \{(0,0),\ (3,5)\}.$$

Neither pair dominates the other so

$$A(1) = \{(0,0),\ (3,5)\}.$$

Iteration 2.

For each existing pair, we generate a new pair by adding $(2,3)$,

$$(0,0) + (2,3) = (2,3)$$
$$(3,5) + (2,3) = (5,8).$$

Taking the union gives

$$A'(2) = \{(0,0),(2,3),(3,5),(5,8)\}.$$

Again, no pair is dominated so set

$$A(2) = A'(2).$$

Iteration 3.

We now generate the following pairs

$$(0,0)+(2,2)=(2,2),$$
$$(2,3)+(2,2)=(4,5),$$
$$(3,5)+(2,2)=(5,7),$$
$$(5,8)+(2,2)=(7,10).$$

Before removing dominated pairs, we have

$$A'(3)=\{(0,0),(2,3),(3,5),(5,8),$$
$$(2,2),(4,5),(5,7),(7,10)\}.$$

Observe thjat

$$(2,3) \text{ dominates } (2,2),$$
$$(3,5) \text{ dominates } (4,5),$$
$$(5,8) \text{ dominates } (5,7).$$

Prune to get

$$A(3)=\{(0,0),(2,3),(3,5),(5,8),(7,10)\}.$$

Iteration 4.

We generate

$$(0,0)+(4,5)=(4,5)$$
$$(2,3)+(4,5)=(6,8)$$
$$(3,5)+(4,5)=(7,10)$$
$$(5,8)+(4,5)=(9,13)$$
$$(7,10)+(4,5)=(11,15).$$

We get

$$A'(4)=\{(0,0),(2,3),(3,5),(5,8),$$
$$(4,5),(6,8),(7,10),(9,13)\}.$$

Notice we do not include $(11,15)$ as $11>B=10$. Here

$$(3,5) \text{ dominates } (4,5)$$
$$(5,8) \text{ dominates } (6,8),$$

so we are left with

$$A(4)=\{(0,0),(2,3),(3,5),(5,8),(7,10),(9,13)\}.$$

Iteration 5.

We generated

$$(0,0)+(3,4)=(3,4)$$
$$(2,3)+(3,4)=(5,7)$$
$$(3,5)+(3,4)=(6,9)$$
$$(5,8)+(3,4)=(8,12)$$
$$(7,10)+(3,4)=(10,14)$$
$$(9,13)+(3,4)=(12,17).$$

We discard $(12,17)$ and get

$$A'(i)=\{(0,0),(2,3),(3,5),(5,8),(7,10),(9,13)$$
$$(3,4),(5,7),(6,9),(8,12),(10,14)\}.$$

Here

$$(3,5) \text{ dominates } (3,4)$$
$$(5,8) \text{ dominates } (5,7),$$

leaving

$$A(5)=\{(0,0),(2,3),(3,5),(5,8),(7,10),(6,9),(8,12),(10,14)\}.$$

Iteration 6.

We generate

$$(0,0)+(1,2)=(1,2)$$
$$(2,3)+(1,2)=(3,5)$$
$$(3,5)+(1,2)=(4,7)$$
$$(5,8)+(1,2)=(6,10)$$
$$(7,10)+(1,2)=(8,12)$$
$$(6,9)+(1,2)=(7,11)$$
$$(8,12)+(1,2)=(9,14)$$
$$(9,13)+(1,2)=(10,15)$$
$$(10,14)+(1,2)=(11,16).$$

From this we get

$$A'(6)=\{(0,0),(2,3),(5,8),(7,10),(6,9),(8,12),(10,14),$$
$$(1,2),(3,5),(4,7),(6,10),(8,12),(7,11),(9,14),(10,15)\}.$$

Here

$$(6,10) \text{ dominates } (7,10)$$
$$(9,14) \text{ dominates } (10,14)$$
$$(6,10) \text{ dominates } (6,9).$$

Thus OPT $=15$. One optimal selection consists of $1,2,4,6$ giving $\sum s=10$, and $\sum v=15$. ■

Statement and proof of (b). A pair (s, v) dominates (s', v'), writing $(s', v') \prec (s, v)$, if

$$s \leq s', \text{ and } v \geq v'$$

with at least one inequality strict.

We claim that $|A(i)| \leq B + 1$.

Every pair $(s, v) \in A(i)$ represents a feasible selection of items. Since item sizes are nonnegative integers and the knapsack has capacity B, we have

$$s \in \{0, 1, \ldots, B\}.$$

Suppose, for a contradiction, that

$$|A(i)| > B + 1.$$

By the pigeonhole principle, there must exist two distinct pairs

$$(s, v_1), (s, v_2) \in A(i)$$

with the same size s.

Since the pairs are distinct, $v_1 \neq v_2$, assume, without loss of generality, that $v_1 > v_2$. Then

$$(s, v_1) \succ (s, v_2),$$

contradicting the definition $A(i)$ as a set of nondominated pairs.

Therefore

$$|A(i)| \leq B + 1.$$

■

Statement and proof of (c). We claim that

$$|A(i)| \leq V + 1.$$

Let

$$V = \sum_{j=1}^{n} v_j.$$

Since all item values are nonnegative integers, every pair $(s, v) \in A(i)$ satisfies

$$v \in \{0, 1, \ldots, V\}.$$

Suppose, once again, for a contradiction, that

$$|A(i)| > V + 1.$$

By the pigeonhole principle, there exist distinct

$$(s_1, v), (s_2, v) \in A(i)$$

with the same value v.

Without loss of generality, suppose $s_1 < s_2$. Then

$$(s_1, v) \succ (s_2, v),$$

again contradicting the fact that $A(i)$ contains only nondominated pairs.

Consequently,

$$|A(i)| \leq V + 1.$$

■